

JS BOM (Browser Object Model)

1. What is BOM in JavaScript?

- a. BOM stands for *Browser Object Model* in Javascript.
- b. The Browser Object Model (BOM) provides the set of objects by the browser to interact and manipulate the browser window.
- c. BOM allows JavaScript to communicate with the browser and perform tasks like:
 - i. Opening/closing windows
 - ii. Managing browser history
 - iii. Navigating to URLs
 - iv. Interacting with screen properties, etc.
- d. Unlike the Document Object Model (DOM), which deals with HTML elements, BOM focuses on the browser itself.

2. BOM Objects:

- a. The key objects in the Browser Object Model are:
 - i. *window* → The main global object representing the browser window/tab.
 - ii. *document* → Represents the HTML document loaded in the window.
 - iii. *screen* → Provides information about the user's screen (like width, height, etc.)
 - iv. *navigator* → Contains information about the browser (user agent, platform, language, etc.).
 - v. *location* → Represents the current URL and allows navigation to other pages.

- vi. *history* → Allows navigation through the browser history.
- vii. *console* → Allows us to access the debugging console of the browser.

3. What is the Window Object?

- a. The Window Object is the *global object* in the BOM.
- b. All other BOM objects (like document , screen , location , etc.) are its properties. These are the direct children of window objects.
- c. *Window objects and its global objects can access anywhere in the application.*

4. Window Properties (Direct Children):

a. document (*window.document*):

- i. The *document* property refers to the HTML document loaded in the browser window.
- ii. It can be accessed using *window.document* syntax. It can also be accessed without prefixing the window object.

b. screen (*window.screen*):

- i. The *screen* object in JavaScript is a property of the 'window' object.
- ii. The 'screen' object's properties contain information about the device's screen.
- iii. The screen object can be accessed by using the *window.screen* syntax.

iv. For Example →

```
console.log(window.screen);
```

- Output:

```
▼ Screen {availWidth: 1536, availHeight: 816, width: 1536, height: 864, colorDepth: 24, ...}
  availHeight: 816
  availLeft: 0
  availTop: 0
  availWidth: 1536
  colorDepth: 24
  height: 864
  isExtended: false
  onchange: null
  ▶ orientation: ScreenOrientation {angle: 0, type: 'landscape-primary', onchange: null}
  pixelDepth: 24
  width: 1536
  ▶ [[Prototype]]: Screen
```

c. navigator (**window.navigator**):

- The **navigator** object in JavaScript is used to access the information of the user's browser.
- Using the 'navigator' object. You can get the browser version and name and check whether the cookie is enabled in the browser.
- For Example →

```
console.log(window.navigator);
```

- Output:

```
Navigator {vendorSub: '', productSub: '20030107', vendor: 'Google Inc.',  
maxTouchPoints: 0, scheduling: Scheduling, ...} bom2.js:41
```

iv. Common Properties:

- language** → The language of the browser (e.g., en-US).

2. *online* → Indicates if the browser is online.
3. *platform* → The platform/OS (e.g., Win32 for Windows).
4. *userAgent* → Browser's user agent string.

d. location (**window.location**):

- i. The *location* object in JavaScript provides information about the browser's location, i.e., URLs.
- ii. It is a built-in property of both the window and document objects.
- iii. We can access it using either *window.location* or *document.location*.
- iv. For Example →

```
console.log(window.location);
```

• Output:

```
Location {ancestorOrigins: DOMStringList, href: 'http://127.0.0.1:5500/DAY_15_BOM/bom.html', origin: 'http://127.0.0.1:5500', protocol: 'http:', host: '127.0.0.1:5500', ...}
  ▶ ancestorOrigins: DOMStringList {length: 0}
  ▶ assign: f assign()
    hash: ""
    host: "127.0.0.1:5500"
    hostname: "127.0.0.1"
    href: "http://127.0.0.1:5500/DAY_15_BOM/bom.html"
    origin: "http://127.0.0.1:5500"
    pathname: "/DAY_15_BOM/bom.html"
    port: "5500"
    protocol: "http:"
  ▶ reload: f reload()
  ▶ replace: f replace()
    search: ""
  ▶ toString: f toString()
  ▶ valueOf: f valueOf()
    Symbol(Symbol.toPrimitive): undefined
  ▶ [[Prototype]]: Location
```

v. **JavaScript Location Object Properties :**

1. *hash* → This property is used to set or get the anchor part of the URL.
2. *host* → This property is used to set or get the hostname or port number of the URL.
3. *hostname* → This property is used to set the hostname.
4. *href* → This property is used to set or get the URL of the current window.
5. *origin* → This property returns the protocol, domain, and port of the URL.
6. *pathname* → This property updates or gets the path name.
7. *port* → This property updates or gets the port of the URL.
8. *protocol* → This property updates or gets the protocol.
9. *search* → This property is used to set or get the query string of the URL.

e. **history (window.history):**

- i. In JavaScript, the window ' *history* ' object contains information about the browser's session history.
- ii. It contains the array of visited URLs in the current session.
- iii. **For Example** →

```
1 console.log(window.history);  
2 // History {length: 1, scrollRestoration: 'auto', state: null}
```

iv. **Common Methods:**

1. `back ()` → The JavaScript `back()` method of the history object loads the previous URL in the history list.
2. `forward ()` → The `forward()` method of the history object takes you to the next URL.
3. `go ( n )` → The `go()` method of the history object takes you to the specific URL of the browser's session.
4. **Syntax** → `go(count)` ⇒ 'count' is a numeric value representing which page you want to visit.

f. **console (window.console):**

- i. In JavaScript, the ' `console` ' object is a property of the window object.
- ii. It allows the developers to access the debugging console of the browser.
- iii. In Node.js, the console object is used to interact with the terminal.
- iv. For Example →

```
console.log(window.console);
```

v. **Common Methods:**

1. `log ()` → Logs messages to the console.
2. `error ()` → Logs error messages.
3. `warn ()` → Logs warnings.
4. `info ()` → Logs informational messages.

5. `count ()` → Counts the number of times a line has been executed.
6. `table ()` → Displays data as a table.
7. `clear ()` → Clears the console.
8. **For Example** →

```
1 console.log(window.console);
2 console.log("This is log() method.");
3 console.error("Something went wrong.");
4 console.warn("Warning occurred!");
5 console.info("Important Information.");
6
7 let array = [1, 2, 3, 4, 5];
8 console.table(array);
9
10 // counts the number of times console.count()
11 console.count(); // default: 1
12 console.count(); // default: 2
```

• **Output:**

```
This is log() method. bom2.js:18
Something went wrong. bom2.js:19
Warning occurred! bom2.js:20
Important Information. bom2.js:21
bom2.js:24


| (index) | Value |
|---------|-------|
| 0       | 1     |
| 1       | 2     |
| 2       | 3     |
| 3       | 4     |
| 4       | 5     |


Array(5)
default: 1 bom2.js:27
default: 2 bom2.js:28
```

5. Window Methods:

a. `window.alert()`:

- i. The `window.alert()` method allows you to show the pop-up dialog containing the message, warning, etc.
- ii. It takes the string text as an argument.
- iii. **For Example** →

```
window.alert("This is an alert!");
```

b. **window.confirm():**

- i. Displays a confirmation box with "OK (*true*)" and "Cancel (*false*)".

- ii. **For Example** →

```
let result = window.confirm("Do you agree?");  
console.log(result); // true(ok), false(cancel)
```

c. **window.prompt():**

- i. Displays a dialog box that prompts the user for input.

- ii. **Syntax** →



```
window.prompt(text, default)  
OR  
prompt()
```

- iii. **For Example** →

```
let name = window.prompt("What is your name?", "John");  
console.log(name);
```

d. **window.open():**

- i. Open a new browser window or tab.

- ii. **Syntax** →



```
window.open(url, target)
```

- iii. **For Example** →

```
window.open("https://www.google.com/", "_blank");
```

e. **window.close():**

- i. Closes the current window (*only works if the window was opened using JavaScript*).

- ii. **For Example** → `window.close();`